



SAPIENZA
UNIVERSITÀ DI ROMA

TTAISS: Tenstorrent AI Similarity Search

Faculty of Information Engineering, Computer Science and Statistics
Master's Degree in Computer Science

Barbalata Radu Ionut

ID number 2002688

Advisor

Prof. De Sensi Daniele

Co-Advisor

PhD student Piarulli Lorenzo

Academic Year 2026/2027

TTAISS: Tenstorrent AI Similarity Search

Master's Thesis. Sapienza University of Rome

© 2026 Barbalata Radu Ionut. All rights reserved

This thesis has been typeset by $\text{\LaTeX}$ and the Sapthesis class.

Author's email: Barbalata.2002688@studenti.uniroma1.it

Abstract

This thesis investigates approximate nearest-neighbor search as a new application domain for the Tenstorrent architecture, with the goal of evaluating whether it can achieve competitive recall while reducing whole-system energy consumption compared with conventional CPU- and GPU-based systems. First, the main families of approximate nearest-neighbor algorithms are analyzed to identify the index structure that best matches Tenstorrent’s spatial architecture and programming model. Based on this analysis, an IVF-Flat search pipeline is designed and implemented on Tenstorrent hardware. The implementation is then evaluated in terms of queries per second, Recall@10, whole-system energy consumption, and infrastructure cost, and compared with multicore CPU configurations and an NVIDIA A100 GPU system. The results show that the GPU achieves the highest throughput across the evaluated recall range, while the Tenstorrent implementation becomes the most energy-efficient measured configuration at the highest evaluated recall point. A simplified hardware-acquisition and electricity-cost model further identifies the operating regions in which Tenstorrent offers a cost advantage over the CPU and GPU baselines. These findings indicate that Tenstorrent is a promising architecture for high-recall ANN workloads, although further performance improvements are currently limited by the maturity and flexibility of its low-level kernel APIs.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	2
1.3	Research Questions	3
1.4	Contributions	3
1.5	Thesis Organization	4
2	Background	5
2.1	Approximate Nearest-Neighbor Search	5
2.2	Inverted File Indexes	6
2.3	Graph-Based Indexes	9
2.3.1	HNSW: Hierarchical Navigable Small World	9
2.3.2	NGT: Neighborhood Graph and Tree	11
2.4	GPU-Specialized ANN Indexes	13
2.4.1	GPU IVF and Faiss	13
2.4.2	GPU Graph Search and CAGRA	14
2.5	Tenstorrent Architecture and Programming Model	15
2.6	Suitability of ANN Indexes for Tenstorrent	17
2.6.1	IVF-Flat	18
2.6.2	Graph-Based Indexes	19
2.6.3	CAGRA	19
2.6.4	IVF-PQ and Quantized Indexes	21
2.6.5	ScaNN	22
2.7	Motivation for Selecting IVF-Flat	23
3	Design and Implementation	26
3.1	System Architecture	26
3.1.1	End-to-End Search Pipeline	26
3.1.2	Host–Device Work Partitioning	28
3.2	Device-Oriented IVF Representation	28
3.2.1	Index Construction	28
3.2.2	Tiled Data Layout and Memory Placement	30
3.3	Coarse Search	32
3.4	Fine-Search Scheduling	33
3.4.1	Batch-Level List Union	33
3.4.2	Greedy Work Distribution	34

3.4.3	Worker Description	35
3.5	Fine Search and Result Reduction	36
3.5.1	Worker Pipeline	36
3.5.2	Hierarchical Top- k	37
3.6	Multi-Batch Execution	39
3.7	Implementation Constraints	39
3.8	Chapter Summary	41
Acknowledgements		42
Bibliography		43

Chapter 1

Introduction

1.1 Motivation

Modern artificial-intelligence systems increasingly rely on external retrieval components to access information that is not contained directly in a model’s parameters. Retrieval-Augmented Generation, or RAG, is a common example: before generating an answer, the system retrieves documents or document fragments that are relevant to the input query and provides them as additional context to the language model.

The retrieval stage is commonly implemented using a vector database. Documents are transformed into a dataset of high-dimensional embedding vectors

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad (1.1)$$

while an incoming query is encoded as a vector $\mathbf{q} \in \mathbb{R}^d$ in the same embedding space. The database then searches for the k vectors in $\mathcal{D}$ that are most similar to $\mathbf{q}$. These vectors correspond to the documents that are expected to contain the most relevant information.

At large scale, vector retrieval can become an important systems bottleneck. A production service may need to process large numbers of queries over datasets containing millions or billions of vectors. Exhaustively comparing every query with every stored vector provides exact results, but its cost grows linearly with the dataset size. Approximate nearest-neighbor indexes reduce this cost by examining only a subset of the indexed vectors, trading a limited amount of recall for higher throughput and lower latency.

Most existing vector-search systems are optimized for conventional CPUs and GPUs. CPUs are effective for irregular graph traversal and branch-heavy algorithms, while GPUs provide high throughput for regular batched computations. However, both architectures may consume substantial power when serving large retrieval workloads. This motivates the investigation of alternative hardware architectures that can provide competitive search quality and performance with lower whole-system energy consumption.

Tenstorrent processors are spatial many-core accelerators designed primarily for machine-learning workloads. They consist of grids of programmable Tensix cores connected through Networks-on-Chip and rely on explicitly managed data movement and distributed local memory [11]. These characteristics make Tenstorrent promising

for regular and highly parallel operations, but it is not immediately clear whether they are also suitable for approximate nearest-neighbor search.

This thesis investigates whether vector retrieval can be efficiently implemented on Tenstorrent hardware. The broader motivation is to evaluate whether the same energy-efficient architecture used for machine-learning inference can also support the retrieval stage of AI systems such as RAG. A successful implementation could reduce the need to execute retrieval and model inference on separate classes of hardware and could provide a lower-energy platform for large-scale vector search.

1.2 Problem Statement

Given the dataset $\mathcal{D}$ and query vector $\mathbf{q}$ introduced above, the nearest-neighbor search problem consists of retrieving the k vectors in $\mathcal{D}$ that are closest to $\mathbf{q}$ according to a chosen distance or similarity measure.

An exact implementation evaluates the distance between the query and every vector in the dataset. For a dataset of n vectors of dimension d , its computational complexity is $O(nd)$ per query, excluding the final top- k selection. Although this computation is regular and highly parallel, its cost becomes significant as n grows.

Approximate nearest-neighbor search reduces the number of evaluated vectors by using an index that identifies a smaller candidate set

$$\mathcal{D}_{\text{candidate}}(\mathbf{q}) \subset \mathcal{D}. \quad (1.2)$$

The final results are selected only from this candidate set. Consequently, some of the true nearest neighbors may be excluded before their distance from the query is evaluated. The central trade-off is therefore between search quality and execution cost: searching more candidates generally improves recall but also increases latency, memory traffic, energy consumption, and hardware utilization.

A formal definition of the exact and approximate versions of the nearest-neighbor search problem, together with the notation and recall metric used throughout the thesis, is given in Section 2.1.

The problem addressed by this thesis is the design and evaluation of an approximate nearest-neighbor search system for Tenstorrent hardware. The implementation must account for the architectural properties of Tensix cores, including tile-based computation, explicitly managed DRAM transfers, limited per-core local memory, distributed execution, and synchronization over the Network-on-Chip.

The thesis focuses on the following challenges:

- identifying an ANN index whose computation and memory-access patterns are compatible with Tenstorrent hardware;
- mapping coarse search, fine search, candidate selection, and top- k processing onto Tensix cores;
- organizing queries and indexed vectors to exploit tile-oriented batch execution;
- reducing irregular memory accesses and unnecessary host-device communication;

- evaluating the resulting trade-off among Recall@10, throughput, energy consumption, and infrastructure cost;
- identifying the main bottlenecks that limit the performance of the current implementation.

1.3 Research Questions

This thesis is guided by the following research questions:

1. Which approximate nearest-neighbor index structure is most suitable for the current Tenstorrent architecture and programming model?
2. Can an IVF-based ANN search pipeline be implemented efficiently using tile-oriented computation, explicit data movement, and distributed Tensix cores?
3. What Recall@10 and throughput can the Tenstorrent implementation achieve compared with multicore CPU systems and an NVIDIA A100 GPU system?
4. How does the whole-system energy consumption of Tenstorrent compare with CPU- and GPU-based ANN implementations at approximately matched recall?
5. Which stages of the Tenstorrent search pipeline dominate execution time, and which architectural or software limitations are responsible for the observed bottlenecks?
6. Under a simplified hardware-acquisition and electricity-cost model, in which operating regions can Tenstorrent provide a cost advantage over conventional CPU and GPU systems?

1.4 Contributions

The main contributions of this thesis are:

1. An analysis of the suitability of major ANN index families for the Tenstorrent architecture, including IVF, graph-based methods, CAGRA, IVF-PQ, and ScaNN.
2. The design and implementation of an IVF-Flat search pipeline on Tenstorrent hardware, including coarse centroid search, cluster selection, distributed fine search, and result processing.
3. A Tenstorrent-specific execution strategy that organizes work around batches of queries and selected cluster sets rather than independent query-cluster tasks.
4. A quality-normalized experimental comparison against two multicore CPU configurations and an NVIDIA A100 GPU system.

5. A whole-system energy and electricity-cost analysis across several recall operating points.
6. A stage-level decomposition of the Tenstorrent search path, showing the relative contribution of host preparation, host-to-device transfers, coarse search, fine search, and output processing.
7. An identification of the main performance limitations of the implementation and a discussion of possible future optimizations and alternative ANN designs.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces approximate nearest-neighbor search, reviews the main index families, describes GPU-specialized approaches, and motivates the selection of IVF-Flat for Tenstorrent. The following chapter presents the system design and implementation, including data layout, coarse search, fine search, scheduling, and device-side execution. The evaluation chapter describes the experimental setup and compares the CPU, GPU, and Tenstorrent systems in terms of recall, throughput, energy efficiency, and infrastructure cost. It also presents the Tenstorrent stage decomposition and bottleneck analysis. The final chapters discuss the main findings, limitations, future work, and conclusions.

Chapter 2

Background

This chapter provides the background required for the rest of the thesis. It introduces the main families of algorithms used for approximate nearest-neighbor search, reviews GPU-specialized methods, and describes the Tenstorrent architecture and programming model. It then analyzes the suitability of each ANN index family for execution on Tenstorrent hardware, motivating the selection of IVF-Flat for the implementation developed in this thesis.

2.1 Approximate Nearest-Neighbor Search

Approximate nearest-neighbor search, commonly abbreviated as ANN or ANNS, addresses the problem of finding the vectors in a dataset that are most similar to a given query vector without necessarily examining every stored vector [3, 4]. ANN search is widely used in information retrieval, recommendation systems, computer vision, semantic search, and retrieval-augmented generation.

Let

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad (2.1)$$

be a dataset containing n vectors of dimension d , and let $\mathbf{q} \in \mathbb{R}^d$ be a query vector. Given a distance or similarity function $s(\mathbf{q}, \mathbf{x})$, the exact k -nearest-neighbor problem consists of finding the set

$$\text{NN}_k(\mathbf{q}) = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}|=k} \sum_{\mathbf{x} \in \mathcal{S}} s(\mathbf{q}, \mathbf{x}). \quad (2.2)$$

Depending on the application, s may represent the inner product, cosine similarity, or the negative Euclidean distance. Using the negative Euclidean distance makes maximizing s equivalent to minimizing distance, so distance- and similarity-based formulations are used interchangeably throughout this thesis.

For normalized vectors, cosine similarity is equivalent to the inner product. If the dataset vectors are stored as the columns of

$$D = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_n] \in \mathbb{R}^{d \times n}, \quad (2.3)$$

then all query–dataset similarities can be computed as

$$\mathbf{r} = \mathbf{q}^T D \in \mathbb{R}^{1 \times n}. \quad (2.4)$$

The k largest values in $\mathbf{r}$ are subsequently selected. The similarity-computation stage requires $O(nd)$ arithmetic operations. Maintaining the best k elements using a bounded heap requires $O(n \log k)$ time.

Exhaustive search is highly regular and can be implemented efficiently using matrix multiplication. Nevertheless, its cost increases linearly with the number of stored vectors, making approximate indexing attractive for large-scale datasets [3, 4]. Exhaustive search becomes expensive for datasets containing hundreds of millions or billions of high-dimensional vectors, particularly when many queries must be served under limited latency or energy budgets.

ANN indexes reduce the amount of work by searching only a query-dependent subset of the dataset. They therefore trade exactness for lower latency and higher throughput. Search quality is commonly measured through recall. For a query $\mathbf{q}$, Recall@ k can be defined as

$$\text{Recall@}k(\mathbf{q}) = \frac{|\text{ANN}_k(\mathbf{q}) \cap \text{NN}_k(\mathbf{q})|}{k}, \quad (2.5)$$

where $\text{ANN}_k(\mathbf{q})$ is the approximate result and $\text{NN}_k(\mathbf{q})$ is the exact ground truth.

The objective of an ANN system is not simply to minimize the number of visited vectors. Instead, it aims to achieve a favorable trade-off among recall, throughput, latency, memory consumption, index-construction cost, and energy usage. The percentage of the dataset examined depends on the workload and index configuration and is not fixed across ANN algorithms.

2.2 Inverted File Indexes

The inverted file index, usually referred to as IVF, is one of the most widely used index structures for large-scale vector search. It is implemented by systems such as Faiss and can be combined with compression techniques including product quantization [3].

IVF partitions the vector space into regions through a coarse quantizer. The coarse quantizer is commonly trained using k -means clustering, producing a set of centroid vectors

$$\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{N_{\text{list}}}\}. \quad (2.6)$$

Each dataset vector is assigned to its closest centroid:

$$a(\mathbf{x}) = \arg \min_{j \in \{1, \dots, N_{\text{list}}\}} \|\mathbf{x} - \mathbf{c}_j\|_2^2. \quad (2.7)$$

The vectors assigned to centroid $\mathbf{c}_j$ form an inverted list

$$L_j = \{\mathbf{x} \in \mathcal{D} \mid a(\mathbf{x}) = j\}. \quad (2.8)$$

The resulting partition can be interpreted as a Voronoi decomposition of the vector space. Every region contains the points for which a particular centroid is the closest coarse representative. Figure 2.1 illustrates this partition: vectors located inside the same colored region are assigned to the same centroid, represented by the central point of that cell. For example, a vector located in the upper-left region is

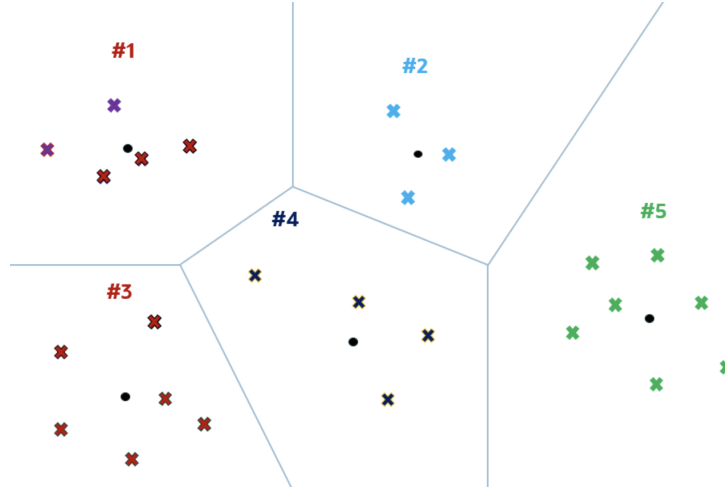


Figure 2.1. Example of a vector space partitioned into Voronoi cells. Each cell is associated with one coarse centroid, and each database vector is assigned to the inverted list of its closest centroid. Adapted from Oracle Database documentation [8].

inserted into the inverted list associated with the upper-left centroid, even when it lies close to the boundary of an adjacent cell.

Search is divided into a coarse quantization stage and a fine-search stage over the selected inverted lists [3]. During coarse search, the query is compared with the N_{list} centroids. The closest N_{probe} centroids are selected. Fine search then evaluates the vectors contained in the corresponding inverted lists:

$$\mathcal{D}_{\text{candidate}}(\mathbf{q}) = \bigcup_{j \in P(\mathbf{q})} L_j, \quad (2.9)$$

where $P(\mathbf{q})$ is the set of lists selected for query $\mathbf{q}$ and

$$|P(\mathbf{q})| = N_{\text{probe}}. \quad (2.10)$$

The main IVF parameters are:

- N_{list} , the number of coarse clusters or inverted lists into which the dataset is partitioned.
- N_{probe} , the number of inverted lists searched for each query.

Increasing N_{probe} generally increases recall because a larger portion of the dataset is examined, but it also increases execution time and memory traffic. Increasing N_{list} produces smaller clusters and can reduce the number of candidates per probed list, although it makes the coarse-search stage more expensive and may produce load imbalance when cluster sizes are not uniform.

The main source of approximation in IVF is the partition-boundary problem. A vector may be very close to the query while belonging to a Voronoi cell whose centroid was not selected during coarse search. In this case, the vector is excluded from the candidate set before its exact distance from the query is computed.

Figure 2.2 shows a concrete example of this behavior. The dataset is partitioned into five Voronoi cells, and the query $\mathbf{q}$ belongs to the red cell. The vectors labelled from #1 to #5 are the five exact nearest neighbors of the query. Four of these vectors belong to the selected red cell and are evaluated during fine search. However, vector #2 lies immediately across the boundary in the neighboring blue cell. Because that cell is not probed, vector #2 is not included in $\mathcal{D}_{\text{candidate}}(\mathbf{q})$, even though it belongs to the exact top-5 result.

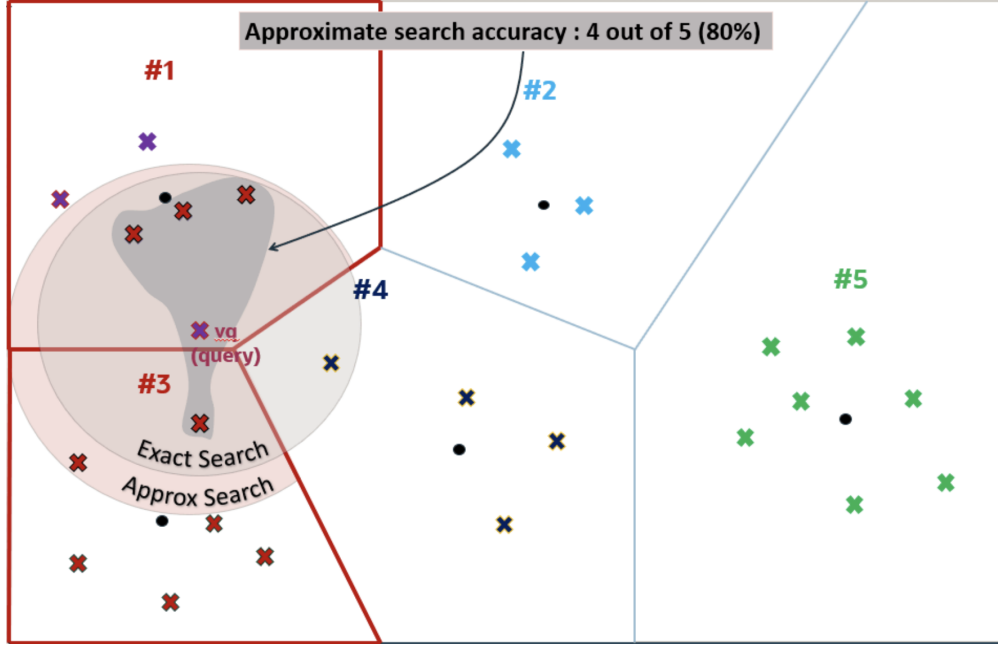


Figure 2.2. Example of a missed nearest neighbor in IVF search. The vectors labelled from #1 to #5 form the exact top-5 result. Vector #2 lies in an unprobed neighboring Voronoi cell and is therefore excluded from the approximate result, which retrieves four of the five exact nearest neighbors.

Using the recall definition introduced in Section 2.1, the approximate result in Figure 2.2 achieves

$$\text{Recall@5} = \frac{|\text{ANN}_5(\mathbf{q}) \cap \text{NN}_5(\mathbf{q})|}{5} = \frac{4}{5} = 0.8. \quad (2.11)$$

The missed vector is not caused by an inaccurate distance calculation. Its distance is never evaluated because coarse search excludes the inverted list containing it. Increasing N_{probe} may include the neighboring blue cell and recover vector #2, but it also increases the number of candidate vectors, distance computations, and memory transfers.

Despite this limitation, IVF exposes substantial parallelism. Coarse distances can be computed independently, selected clusters can be searched concurrently, and distance computation within each cluster is regular. Vectors in an inverted list can also be stored contiguously, providing more predictable memory accesses than pointer-based index structures.

IVF-Flat stores the original vectors inside each inverted list. Consequently, the distances evaluated during fine search are exact with respect to the selected

candidates; approximation is introduced only by the coarse partitioning and the selection of a subset of the lists.

Several extensions reduce the memory footprint or the amount of fine-search work. IVF-PQ stores product-quantized codes instead of complete vectors, while related methods compute approximate distances in the compressed domain [3]. ScaNN combines search-space pruning with quantization techniques, including anisotropic vector quantization for maximum inner-product search [1]. These approaches can significantly reduce memory traffic but introduce additional codebook lookups, approximate-distance computations, and often a re-ranking stage.

2.3 Graph-Based Indexes

Graph-based ANN methods represent dataset vectors as vertices of a proximity graph. Edges connect vectors that are close according to the selected metric. At query time, search starts from one or more entry vertices and repeatedly moves toward neighboring vertices that appear closer to the query.

Unlike IVF, graph search does not define a fixed set of spatial partitions to visit. The visited region is determined dynamically by the distances observed during traversal. This can provide high recall while evaluating a relatively small fraction of the dataset, including points located across coarse partition boundaries.

Graph-based methods are particularly effective on CPUs because CPUs provide efficient branch execution, cache hierarchies optimized for irregular access, and SIMD instructions for computing distances between a query and a small set of candidate vectors. Their main disadvantage is that traversal is data-dependent: different queries visit different vertices, perform different numbers of iterations, and access different memory locations.

The two representative graph-based approaches considered in this chapter are HNSW, described in Section 2.3.1, and NGT, described in Section 2.3.2.

2.3.1 HNSW: Hierarchical Navigable Small World

Hierarchical Navigable Small World, or HNSW, organizes proximity connections into multiple layers [4]. The bottom layer contains all indexed vectors, while progressively higher layers contain increasingly sparse subsets.

The level assigned to each vector is selected probabilistically, with the probability of appearing at higher levels decreasing exponentially. This creates a hierarchy conceptually similar to a skip list: upper layers contain long-range connections that allow the search to move rapidly across the dataset, while lower layers provide increasingly local refinement [9].

Search begins from an entry point in the highest layer. At each level, the algorithm greedily follows neighbors that improve the distance to the query. Once no closer neighbor is found, the search continues from the current vertex in the next lower layer. At the bottom layer, a wider best-first search is performed using a dynamically maintained candidate queue [4].

A simplified representation of the search procedure is:

1. select an entry vertex in the highest graph layer.

2. greedily traverse the current layer toward the query.
3. use the resulting vertex as the entry point for the layer below.
4. at the bottom layer, maintain a bounded set of promising candidates.
5. terminate when the remaining candidates cannot improve the current result set.

Figure 2.3 illustrates this hierarchical organization. The upper layers contain only a sparse subset of the indexed vectors and provide long-range edges that allow the search to move rapidly toward the region containing the query. After reaching a local minimum in one layer, the search descends to the corresponding vertex in the next denser layer. The final and most detailed exploration is performed in layer 0, which contains all indexed vectors. This hierarchy allows HNSW to combine coarse navigation in the upper layers with fine-grained neighbor exploration in the bottom layer.

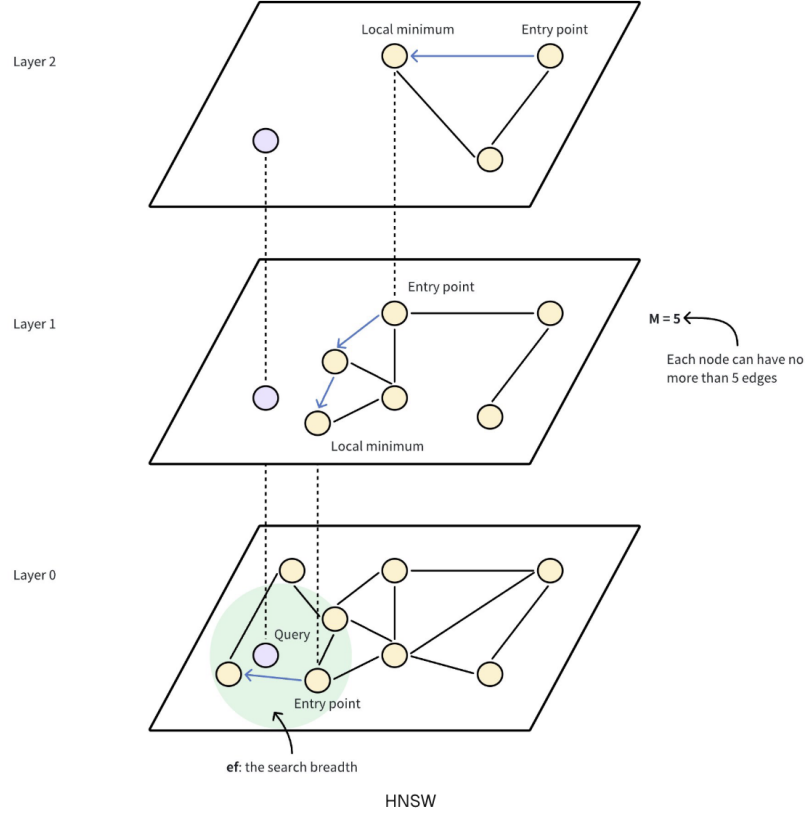


Figure 2.3. Hierarchical organization of an HNSW index. Sparse upper layers provide long-range navigation, while the dense bottom layer supports fine-grained nearest-neighbor search. Adapted from the Milvus HNSW documentation [5].

The maximum number of connections per vertex and the candidate-list size control the memory, recall, and latency trade-offs. HNSW is known for strong high-recall performance but may require substantial memory because both the original

vectors and graph adjacency lists must be stored.

From an accelerator perspective, HNSW presents several challenges:

- neighbor addresses depend on the currently visited vertex.
- adjacency lists and vectors may be distributed across memory.
- the search frontier and result queue are updated dynamically.
- different queries follow different paths and terminate after different numbers of steps.
- visited-set management requires additional random accesses and synchronization.

These characteristics do not prevent accelerator implementations, but they make it harder to obtain the regular, high-throughput execution typically associated with dense tensor operations. Unlike the IVF partitioning described in Section 2.2, HNSW can cross coarse spatial boundaries by traversing graph edges. The consequences of its irregular traversal for Tenstorrent are discussed in Section 2.6.2.

2.3.2 NGT: Neighborhood Graph and Tree

Neighborhood Graph and Tree, or NGT, combines a neighborhood graph with a separate tree-based structure used to obtain good entry points for graph traversal [2]. Conceptually, it is similar to HNSW in that both methods first try to move the search toward a promising region of the vector space before performing a more detailed local graph exploration.

The main difference lies in how this initial region is reached. In HNSW, the search uses the hierarchy of the graph itself: it starts from sparse upper layers and progressively descends toward the denser bottom layer. NGT instead uses a separate tree-based structure to locate one or more objects that are expected to be close to the query. These objects are then used as entry vertices for the neighborhood graph.

In this sense, the tree in NGT plays a role similar to the upper layers of HNSW, since both mechanisms reduce the need to begin graph traversal from an arbitrary point. However, the two approaches are structurally different. HNSW integrates coarse navigation and local search into a single hierarchical graph, whereas NGT separates the initial tree-based localization from the subsequent neighborhood-graph traversal.

Once the entry vertices have been identified, NGT explores the neighborhood graph by maintaining a candidate set ordered according to distance from the query. The most promising unexplored candidate is expanded, its neighboring vertices are evaluated, and sufficiently close neighbors are inserted into the candidate set. This process continues until the termination condition is satisfied, after which the best k objects found during traversal are returned.

Figure 2.4 summarizes the search process. The tree first performs a coarse localization of the query and selects the initial graph vertices. The neighborhood graph is then explored from these seeds to progressively refine the candidate set and obtain the approximate nearest neighbors.

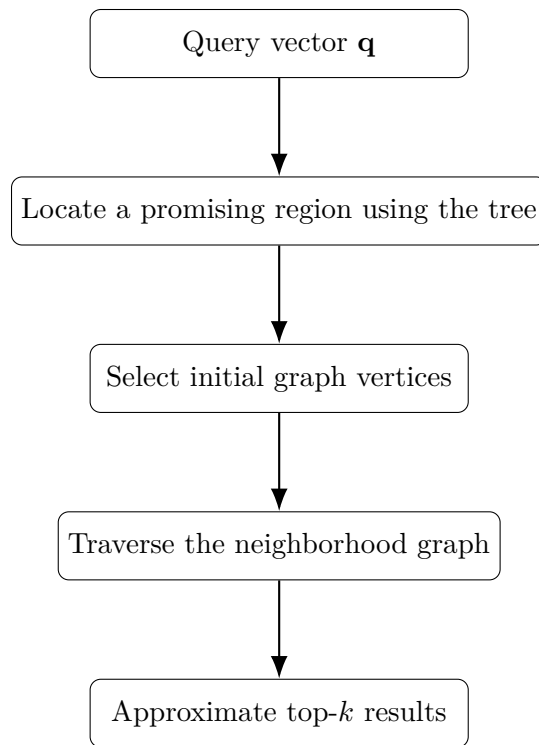


Figure 2.4. High-level NGT search process. The tree first identifies a promising region of the vector space and provides initial graph vertices. The neighborhood graph is then explored from these seeds to refine the search and obtain the approximate nearest neighbors. Original illustration based on [2].

As with HNSW, NGT can achieve high recall while evaluating only a subset of the dataset. Its execution is nevertheless irregular: the next vertices to visit depend on the query and on the distances observed during traversal. The search therefore involves non-contiguous graph accesses, dynamic candidate-set updates, and branch-dependent control flow. The additional tree traversal introduces another query-dependent stage before neighborhood exploration begins.

For this thesis, HNSW and NGT illustrate a common property of graph-based ANN methods: they reduce the number of distance computations by relying on irregular memory accesses and dynamic traversal decisions. These characteristics differ substantially from the regular, tile-oriented execution model targeted by the Tenstorrent architecture, and their suitability is discussed jointly in Section 2.6.2.

2.4 GPU-Specialized ANN Indexes

GPUs provide high arithmetic throughput and memory bandwidth, making them well-suited to regular batched distance computations. However, efficient ANN search on GPUs requires algorithms that expose sufficient parallelism and avoid frequent divergent branches or fine-grained dynamic memory operations.

2.4.1 GPU IVF and Faiss

IVF naturally maps to GPU execution because many queries, clusters, and candidate vectors can be processed concurrently. GPU implementations can parallelize both the coarse centroid comparisons and the fine-search distance calculations over selected clusters.

One important difficulty is selecting the smallest or largest k elements from a large set of candidate distances. A conventional heap performs data-dependent insertions and provides insufficient parallelism for efficient GPU execution.

The GPU implementation presented by Johnson et al. in Faiss introduces a register-resident selection method designed around GPU warps [3]. Candidate values are processed cooperatively by the threads of a warp, reducing global-memory traffic and avoiding the use of a conventional branch-heavy heap. The method is integrated with brute-force, IVF, and product-quantized search.

The resulting search pipeline uses the GPU effectively because:

- distance calculations are independent and data parallel.
- the same kernel can process large batches of queries.
- candidate vectors can be read through coalesced or block-oriented memory accesses.
- selection is performed using warp-level cooperation.
- intermediate results can remain in registers or shared memory.

The Faiss work demonstrates that the top- k stage, rather than only distance computation, must be designed specifically for the target architecture [3].

2.4.2 GPU Graph Search and CAGRA

Traditional proximity-graph indexes are difficult to execute efficiently on GPUs because each query follows a data-dependent path and maintains its own candidate and visited sets. CAGRA addresses this problem through a graph structure and traversal strategy designed specifically for highly parallel GPU execution [7].

CAGRA demonstrates that graph-based ANN search can be adapted to GPUs, but this requires substantially more than transferring a conventional CPU graph traversal to CUDA. Its design modifies both graph construction and query execution to expose bounded and predictable parallel work.

First, CAGRA constructs a single-layer proximity graph with a fixed out-degree, bounding the amount of work associated with each vertex expansion [7]. A bounded degree ensures that expanding one graph vertex always produces a known maximum number of neighbors, allowing their vector distances to be evaluated cooperatively by GPU threads. This is considerably more regular than a graph in which adjacency-list lengths vary arbitrarily.

Second, graph traversal is organized around fixed-capacity candidate and result structures rather than dynamically allocated CPU priority queues. Threads within a GPU block cooperate in expanding candidates, computing distances, filtering previously visited vertices, and updating the candidate set. These structures must fit within the memory and synchronization model available to a GPU thread block.

Third, visited-node tracking is implemented using a bounded GPU-oriented structure. Unlike an exact dynamically growing CPU set, this representation accepts limited capacity and possible repeated visits in exchange for predictable memory usage and parallel lookup behavior. The search procedure is therefore designed to tolerate approximation not only in graph traversal but also in auxiliary bookkeeping.

Finally, the stopping policy and the number of expanded candidates are adapted to parallel execution. The algorithm must keep enough work available for the GPU while limiting divergent control flow between queries. These changes show that accelerator graph search requires the graph degree, candidate queue, visited set, work assignment, and termination condition to be co-designed with the target architecture [7].

CAGRA versus IVF on GPUs

Neither CAGRA nor IVF is universally superior on GPUs. The preferable index depends on the required recall, dataset size and distribution, query-batch size, available memory, and index-construction constraints.

IVF exposes regular and highly parallel execution because vectors within each selected inverted list can be read and evaluated in contiguous blocks. Its index structure is comparatively simple, and quantized variants such as IVF-PQ can reduce storage and device-memory bandwidth. However, high-recall search may require probing many inverted lists. As N_{probe} increases, the system evaluates more candidate vectors and approaches the cost of exhaustive search.

CAGRA instead uses graph connectivity to navigate toward promising areas of the dataset without scanning entire clusters. Its fixed-degree graph and cooperative GPU traversal are designed to provide high throughput at high recall [7]. This

advantage comes with additional graph storage, more complex index construction, indirect vector accesses, and candidate and visited-state structures that must be managed during traversal.

CAGRA is therefore particularly attractive for high-recall, memory-resident GPU workloads when graph construction and storage costs are acceptable. IVF remains attractive when regular execution, simpler construction, predictable partitioning, support for datasets larger than available accelerator memory, or compatibility with compression are more important. The comparison is implementation-dependent: optimized quantized IVF variants may outperform graph-based methods at some high-recall operating points [10].

The approach used by CAGRA is highly relevant when considering graph search on Tenstorrent. It shows that accelerator execution is possible, but a direct port of a CPU graph algorithm is unlikely to be efficient. The graph structure, memory layout, candidate management, and traversal strategy must all be redesigned around the target accelerator.

2.5 Tenstorrent Architecture and Programming Model

Tenstorrent processors are spatial many-core accelerators composed of a two-dimensional grid of Tensix cores connected through one or more Networks-on-Chip. Rather than relying primarily on hardware-managed caches and general-purpose threads, the programming model gives the developer explicit control over data placement, data movement, core assignment, and synchronization.

Figure 2.5 shows the main components of a Tensix core. A core combines local L1 SRAM, RISC-V processors responsible for data movement and kernel control, and specialized compute engines. The data-movement processors issue transfers through the NoC, while the compute path includes a matrix engine for tile-oriented operations and a vector engine for element-wise and special-function operations. The local SRAM stores circular buffers, input and output tiles, intermediate values, runtime metadata, and other kernel state. Consequently, performance depends not only on arithmetic throughput but also on how effectively computation and data movement are pipelined through the limited local-memory capacity.

Tenstorrent provides several programming layers with different levels of abstraction. At the highest level considered here, TT-NN provides a Python interface and a library of common tensor and machine-learning operations. It is suitable when an application can be expressed using operations already provided by the library, allowing the runtime to manage much of the device configuration and tensor placement.

TT-Lang provides an intermediate programming layer between TT-NN and TT-Metalium. It is a Python-based domain-specific language for implementing custom Tenstorrent operations while retaining explicit control over data movement, synchronization, and inter-core communication. It provides higher-level constructs such as tensor slices, blocks, and pipes, while remaining more flexible than the predefined operations available in TT-NN. TT-Lang was under active development during this work, and the implementation presented in this thesis was developed directly using TT-Metalium.

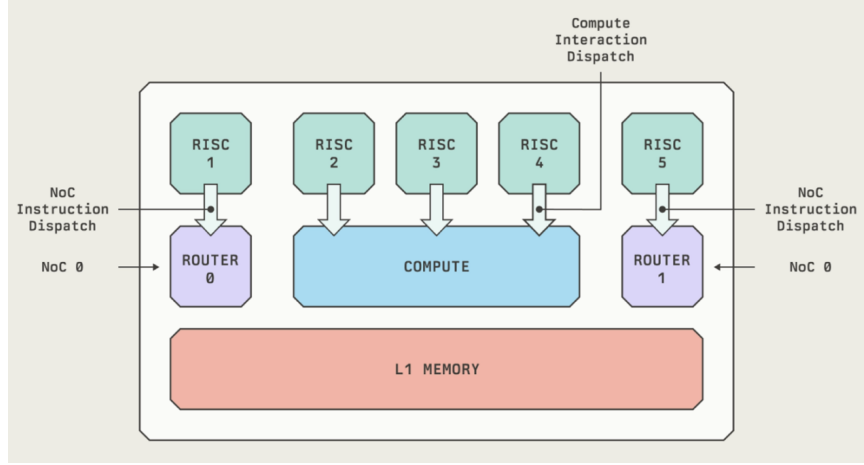


Figure 2.5. Simplified organization of a Tensix core, including its RISC-V control and data-movement processors, local L1 SRAM, NoC interfaces, and specialized compute engines [11].

TT-Metalium is the lower-level C++ programming interface used to define programs, allocate buffers, select Tensix cores, configure circular buffers, compile kernels, set runtime arguments, and explicitly schedule data movement. TT-Metalium exposes the architecture in sufficient detail to implement non-standard operations such as the ANN kernels developed in this thesis. A typical operation is built from reader, compute, and writer kernels and is first developed for one Tensix core before being distributed across multiple cores or devices.

TT-LLK, or the Low-Level Kernel layer, contains architecture-specific implementations and interfaces for the underlying Tensix compute engines. TT-Metalium compute APIs are commonly built on these lower-level kernels, while direct modification or extension of TT-LLK may be required when the available operations do not expose a needed hardware primitive. In this thesis, the limitations of the available top- k , comparison, and data-manipulation primitives influence which ANN algorithms can be implemented efficiently.

The software stack can therefore be viewed as

$$\text{TT-NN} \rightarrow \text{TT-Lang} \rightarrow \text{TT-Metalium} \rightarrow \text{TT-LLK} \rightarrow \text{Tensix hardware}, \quad (2.12)$$

where moving downward provides greater architectural control but also requires more explicit management of memory, synchronization, tilization, and kernel execution.

Most computation on a Tensix core is performed on tiles. The standard tile is a 32×32 block whose elements share the same data type. Tensors whose final dimensions are not multiples of the tile dimensions generally require padding or a different supported layout.

The relevant memory hierarchy includes:

- off-chip device DRAM, which provides the capacity required for large tensors and indexes.

- local L1 SRAM associated with each Tensix core, which provides lower-latency storage for tiles, intermediate values, and circular buffers.
- explicit transfers over the Network-on-Chip between DRAM interfaces and Tensix cores or among Tensix cores.

Data in DRAM is not accessed through a transparent cache. Device kernels explicitly request transfers using the NoC data-movement APIs. Therefore, efficient execution depends strongly on the regularity of the memory-access pattern, the size of each transfer, and the placement of data relative to the cores that consume it. A typical TT-Metalium program uses three kinds of kernels:

- a reader kernel, responsible for transferring input data into local buffers.
- a compute kernel, which operates on tiles using the Tensix compute engine.
- a writer kernel, responsible for transferring results to their final location.

These kernels communicate through circular buffers in local SRAM. Circular buffers implement a producer-consumer pipeline: a reader reserves space and pushes input tiles, the compute kernel waits for available tiles and produces outputs, and the writer waits for output tiles before transferring them. Double buffering can overlap data movement with computation.

This execution model is particularly effective for workloads with:

- regular tile-based computation.
- statically partitionable work.
- predictable data movement.
- contiguous or block-oriented memory accesses.
- sufficient work per transferred tile.
- opportunities for batching and data reuse.

Conversely, workloads dominated by pointer chasing, fine-grained random access, dynamically sized data structures, and query-dependent control flow are more difficult to map efficiently. Such workloads may still be implementable, but often require transformations that increase the amount of work, memory traffic, or synchronization.

2.6 Suitability of ANN Indexes for Tenstorrent

The suitability of an ANN index for Tenstorrent depends on more than its algorithmic complexity. An index that performs fewer distance computations may still execute inefficiently if it requires irregular memory accesses, dynamic priority queues, or frequent synchronization.

The principal criteria considered in this thesis are:

1. regularity of the computation.

2. spatial locality of the indexed vectors.
3. predictability and granularity of DRAM transfers.
4. ability to batch multiple queries.
5. amount of query-dependent control flow.
6. size and dynamism of intermediate data structures.
7. possibility of statically distributing work across Tensix cores.

2.6.1 IVF-Flat

IVF-Flat provides the most natural mapping among the considered ANN indexes. Both coarse and fine search are dominated by independent vector-distance calculations. The centroids form a dense matrix, while vectors assigned to the same cluster can be stored in contiguous regions of device DRAM.

Coarse search can be performed for a batch of queries by comparing a 32-query tile with tiles containing the centroid vectors. This arrangement matches the native tile dimensions and allows the queries to be reused while centroid blocks are streamed through the compute cores.

Fine search can then be partitioned by assigning selected clusters or groups of clusters to different workers. Each worker reads contiguous vector blocks, computes their distances from the corresponding queries, and produces a local set of candidates. Partial results are subsequently merged.

The implementation developed in this thesis does not treat each query-cluster pair as a completely independent GPU-style task. Instead, work is organized around batches of up to 32 queries and the union of the clusters selected for those queries. This allows the data layout and computation to remain compatible with tile-based execution.

The main difficulties are:

- inverted lists have different numbers of vectors.
- different queries may select overlapping or disjoint sets of lists.
- the amount of work increases with N_{probe} .
- workers may receive unequal candidate-set sizes.
- local candidate sets must be merged into a global top- k .
- cluster metadata and worker schedules must be generated and transferred efficiently.

These issues require explicit scheduling and load balancing, but the dominant computation remains regular enough to benefit from the Tensix architecture. As discussed in Section 2.7, this combination of properties made IVF-Flat the index selected for the implementation developed in this thesis.

This section provides only the architectural motivation for mapping IVF-Flat to Tenstorrent. The complete implementation, including the data layout, query

batching, coarse-search kernels, fine-search scheduling, top- k processing, and host-device interaction, is described in Chapter 3.

2.6.2 Graph-Based Indexes

Graph-based indexes are less naturally aligned with the current Tenstorrent programming model. To expand one graph vertex, the implementation must first retrieve its adjacency list and then fetch the vectors associated with the listed neighbors. The addresses of these vectors are known only after the current vertex has been processed.

The resulting memory-access sequence is therefore indirect:

$$\text{vertex} \rightarrow \text{adjacency list} \rightarrow \text{neighbor identifiers} \rightarrow \text{neighbor vectors}. \quad (2.13)$$

The neighbor vectors are not guaranteed to be contiguous in DRAM. Transferring complete tiles may consequently fetch vectors that are not needed by the current traversal. Alternatively, issuing small row-major transfers can reduce the transferred volume but may underutilize DRAM and NoC bandwidth.

Graph traversal also requires:

- a dynamic candidate queue ordered by distance.
- a bounded result set.
- a representation of the previously visited vertices.
- repeated termination checks.
- synchronization when multiple workers cooperate on one query.

Batching does not completely solve the problem because different queries may visit unrelated regions and execute different numbers of iterations. Some queries may therefore finish while others continue traversing the graph, reducing the utilization of cores assigned to the batch.

Graph indexes should therefore not be described as impossible to implement on Tenstorrent. A possible implementation could use fixed-degree graphs, fixed-capacity candidate buffers, batched frontier expansion, and approximate visited sets. However, this would require a substantial redesign similar in spirit to CAGRA and was beyond the scope of this thesis.

2.6.3 CAGRA

CAGRA is designed for GPU execution, but its GPU-oriented design does not automatically make it efficient on Tenstorrent. Both architectures exploit parallelism, but their execution models and memory hierarchies differ.

The fixed graph degree in CAGRA is beneficial because it bounds the amount of work associated with one vertex expansion. Groups of 32 neighbors could, in principle, be arranged to match a Tensix tile dimension. Nevertheless, the identifiers still refer to vectors that may be located in unrelated DRAM regions. Efficiently gathering those vectors remains difficult.

The candidate queue and visited-state structure also require careful consideration. A GPU implementation can use registers, shared memory, and thread-block cooperation to maintain bounded structures. On Tenstorrent, each Tensix core has a limited amount of L1 SRAM that is also used for circular buffers, input tiles, output tiles, metadata, and program state. Reserving a large per-query candidate or visited structure would reduce the space available for double buffering and data reuse.

Placing these structures in device DRAM would increase capacity but introduce additional NoC transfers and latency on every graph iteration. Distributing them over multiple Tensix cores would require synchronization and ownership rules for concurrent updates.

A Tenstorrent-oriented graph index could still be explored in future work, for example by combining:

- a fixed graph degree compatible with tile dimensions.
- batched expansion of several query frontiers.
- a compact probabilistic visited-state representation.
- graph reordering to improve spatial locality.
- bounded iteration counts and fixed-capacity candidate queues.

Another possible Tenstorrent-specific design would group multiple dataset vectors into one logical graph vertex. For example, each logical vertex could contain a block of 32 vectors, allowing one block to align with a native tile dimension and enabling the distance calculations for those vectors to be performed together. The graph connecting these blocks could use a small fixed degree, such as four outgoing edges. Most edges could connect blocks representing nearby regions, while one edge could provide a longer-range connection intended to improve global navigability.

This organization would reduce the granularity mismatch between conventional graph traversal and tile-oriented execution. Fetching one graph vertex would provide enough vectors to occupy the compute engine, and a fixed low degree would bound the number of successor blocks considered during each expansion. It could also reduce adjacency metadata compared with representing every vector as an independent graph node.

However, grouping vectors changes the semantics of the graph. The 32 vectors contained in one block may not all be equally close to one another or to the query, and selecting a block may require evaluating many vectors that would not have been visited by a conventional graph search. Constructing balanced blocks with both good internal locality and useful inter-block edges would therefore become a separate indexing problem. The search would still require a candidate queue, a visited-block representation, query-dependent termination, and indirect DRAM accesses to successor blocks. Long-range edges would also need to be constructed carefully to improve navigability without causing excessive exploration. This design may better match Tensix execution than a conventional per-vector graph, but it does not eliminate the fundamental control-flow and memory-access challenges of graph-based ANN search.

Such an implementation would therefore constitute a separate research project rather than a direct extension of the IVF pipeline.

2.6.4 IVF-PQ and Quantized Indexes

Product quantization divides a vector into m sub-vectors and replaces each sub-vector with the identifier of its closest codeword. A database vector is therefore represented using a short sequence of codebook indices rather than its original coordinates. Figure 2.6 illustrates this process: the original vector is divided into independent subspaces, each sub-vector is mapped to a codeword from its corresponding codebook, and the resulting codeword identifiers form the compressed representation.

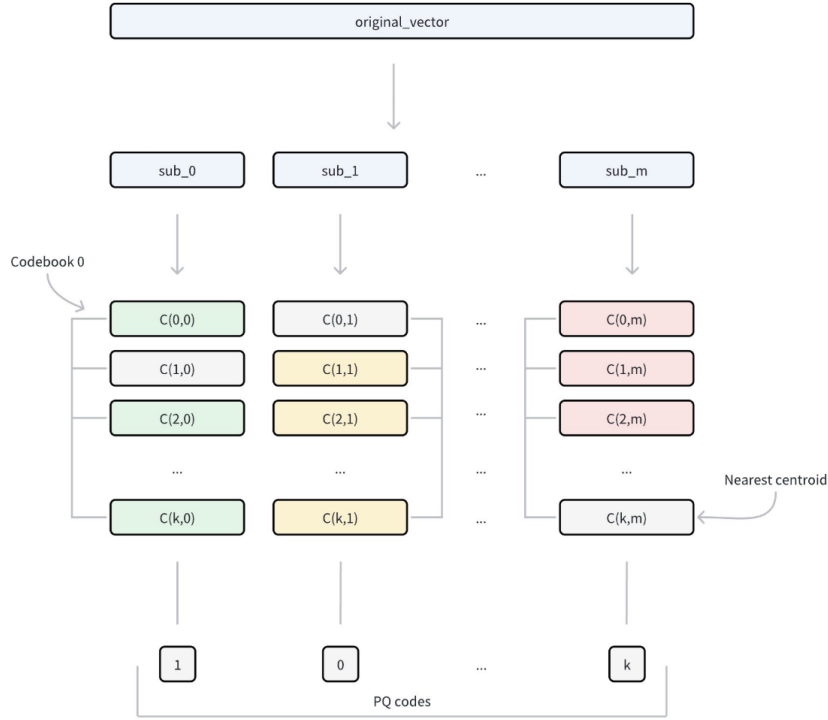


Figure 2.6. Overview of product quantization within an IVF-PQ index. Each vector is divided into sub-vectors and represented by codeword identifiers from independently trained subspace codebooks. Approximate distances are computed by accumulating values from query-specific lookup tables. Adapted from the Milvus IVF-PQ documentation [6].

For a query, lookup tables containing the distance between each query sub-vector and each codeword can be precomputed. As shown in Figure 2.6, the approximate distance to a compressed database vector is obtained by using its codeword identifiers to select one value from each lookup table and summing the selected values. IVF-PQ preserves the coarse partitioning of IVF and therefore retains many of its favorable properties:

- encoded vectors can be stored contiguously within each inverted list.
- many codes can be evaluated independently.
- the reduced representation lowers DRAM capacity and bandwidth requirements.

- coarse search and cluster scheduling can reuse the IVF-Flat design.

The difficulty is not that the original vector must be fetched for every candidate. During the approximate stage, distances can be computed directly from the compressed codes. Original vectors are required only when an optional exact re-ranking stage is performed.

The main implementation challenges on Tenstorrent are:

- implementing efficient small-table lookups.
- decoding packed codeword identifiers.
- supporting integer and mixed-precision accumulation.
- arranging codebooks for efficient local reuse.
- gathering original vectors if exact re-ranking is enabled.

Although IVF-PQ reduces the amount of data stored in DRAM, its fine-search access pattern is less suitable for the current Tenstorrent implementation than the sequential reading of full vectors used by IVF-Flat. Each candidate code contains several packed codeword identifiers, and evaluating it requires decoding those identifiers and performing multiple indexed accesses into relatively small distance tables. These gathers are regular at the algorithmic level but do not directly correspond to the dense tile operations for which the available Tensix compute primitives are optimized.

The lookup tables could be copied into local SRAM and reused across many candidates, but each Tensix core would then need sufficient local capacity for the tables, circular buffers, candidate codes, intermediate distances, and top- k state. If the tables or decoded data cannot remain local, repeated accesses through device memory or the NoC may offset part of the bandwidth reduction obtained from compression. Packed-code extraction and mixed-precision accumulation would also require additional low-level kernels.

For these reasons, IVF-PQ was not considered a promising first index for the current implementation. This conclusion concerns its mapping to the present Tenstorrent software stack rather than the general efficiency of IVF-PQ. A specialized implementation with local lookup-table reuse, efficient packed-code decoding, and dedicated quantized-distance kernels could make IVF-PQ attractive in future work.

2.6.5 ScaNN

ScaNN combines partitioning, quantization, and re-ranking. Its anisotropic quantization objective gives greater importance to preserving components of the quantization residual that have a larger effect on the inner product [1].

A typical ScaNN-style search pipeline combines partition-based pruning, compressed candidate scoring, candidate selection, and optional exact re-ranking [1]:

1. partition-based pruning of the search space.
2. approximate scoring of a comparatively large candidate set using highly compressed representations.

3. selection of a sufficiently large set of promising candidates.
4. re-ranking of those candidates using their original vectors to obtain the final top- k results.

The partitioning and approximate-scoring stages offer parallelism and could potentially be mapped to Tensix cores. However, an efficient implementation would require optimized quantized-distance kernels and a sufficiently large intermediate candidate selection.

In the implementation considered in this thesis, the available top- k primitive from TT-LLK and the local-memory budget constrain the number of candidates that can be maintained efficiently inside one device program. Re-ranking also requires gathering original vectors whose identifiers were produced by the compressed search stage. Unless the dataset is reordered or the selected identifiers exhibit locality, these vectors may be distributed across device DRAM.

These issues do not make ScaNN fundamentally incompatible with Tenstorrent. They indicate that an efficient implementation would require additional support for large candidate selection, quantized lookups, and batched vector gathering. For this reason, ScaNN was considered beyond the scope of the first Tenstorrent ANN implementation.

2.7 Motivation for Selecting IVF-Flat

Section 2.6 compared IVF-Flat, graph-based indexes, CAGRA, IVF-PQ, and ScaNN against the same set of criteria: regularity of computation, spatial locality, DRAM-transfer granularity, batchability, query-dependent control flow, and the possibility of static work distribution. IVF-Flat was the only index for which all of these criteria were favorable without requiring a substantial architectural redesign, which is why it was selected for the implementation developed in this thesis.

Two further points reinforced this choice. First, IVF-Flat avoids compression-related approximation during candidate scoring: once a list has been selected, its vectors are compared with the query using their original representation. This simplifies correctness validation and allows the effects of N_{list} , N_{probe} , scheduling, and data movement to be studied directly without conflating them with quantization error.

Second, IVF still exposes challenges representative of real accelerator systems, including variable cluster sizes, load balancing, host-side scheduling, DRAM placement, NoC traffic, local top- k selection, and result merging, without requiring the fully dynamic execution model of graph-based search.

Table 2.1 summarizes the principal trade-offs of the index families considered in this chapter.

The selection of IVF-Flat should not be interpreted as evidence that other ANN indexes cannot execute on Tenstorrent. Instead, IVF-Flat is the index whose computation, memory layout, and scheduling requirements most directly match the capabilities of the current architecture and software stack. It provides a suitable starting point for studying the feasibility, performance, energy efficiency, and bottlenecks of ANN search on Tenstorrent hardware. Its complete implementation is

Table 2.1. Summary of the ANN indexes considered and their qualitative suitability for the current Tenstorrent programming model.

Index	Main advantage	Main limitation	Tenstorrent suitability
Flat	Regular exhaustive computation	Linear work in dataset size	Regular, but excessively expensive at scale
IVF-Flat	Contiguous candidate scans and tunable recall	May miss vectors in unprobed cells	High; selected for this thesis
HNSW	High recall with relatively few distance evaluations	Irregular traversal and substantial graph state	Low without substantial redesign
NGT	Tree-based seeding followed by graph refinement	Dynamic traversal and indirect memory accesses	Low without substantial redesign
CAGRA	GPU-oriented high-recall graph search	Complex traversal state and indirect accesses	Potentially feasible with a TT-specific redesign
IVF-PQ	Reduced storage and memory traffic	Packed decoding and lookup-table accesses	Moderate, but difficult with current primitives
ScaNN	Effective pruning, quantization, and re-ranking	Large intermediate selection and irregular re-ranking	Moderate to low with the current software stack

presented in Chapter [3](#).

Chapter 3

Design and Implementation

3.1 System Architecture

This chapter presents the system design and implementation of a Tenstorrent-oriented IVF-Flat approximate nearest-neighbour search pipeline on a single-chip Tenstorrent Wormhole n150 device using TT-Metalium [11, 12]. IVF-Flat provides the underlying indexing algorithm; the contribution of this chapter is its mapping to the target accelerator. In particular, the design maps regular, fixed-shape operations, including centroid scoring and page-wise candidate scoring, onto the tile-oriented Tensix execution model, while explicitly handling the irregularity caused by query-dependent list selection and unequal inverted-list sizes. The architectural meaning of regular and irregular execution is introduced in Section 2.6.

The implementation is organised around two principal device programs: a *coarse-search program*, which compares queries with the IVF centroids, and a *fine-search program*, which evaluates candidate vectors from the selected inverted lists and reduces the results to the final top- k . Host-side processing connects the two phases by selecting the N_{probe} lists returned by coarse search and constructing the work schedule used by the fine-search workers.

3.1.1 End-to-End Search Pipeline

A query is first compared with all N_{list} centroids, after which the N_{probe} most similar centroids identify the inverted lists to be searched. Candidate vectors contained in those lists are then compared with the query, and the final top- k scores and vector identifiers are selected from the resulting similarities. This logical IVF-Flat pipeline is summarised in Figure 3.1.

The figure is intentionally logical rather than physical: selected candidates are not copied into a single temporary matrix. Instead, database vectors remain grouped by inverted list in device memory and are streamed directly from the selected complete lists during fine search.

Queries are represented internally in groups containing between one and 32 valid queries, matching the row dimension of the 32×32 tile representation used by the Tensix compute kernels. A group containing fewer than 32 valid queries is completed with zero-valued padded rows. For a workload containing Q valid queries,

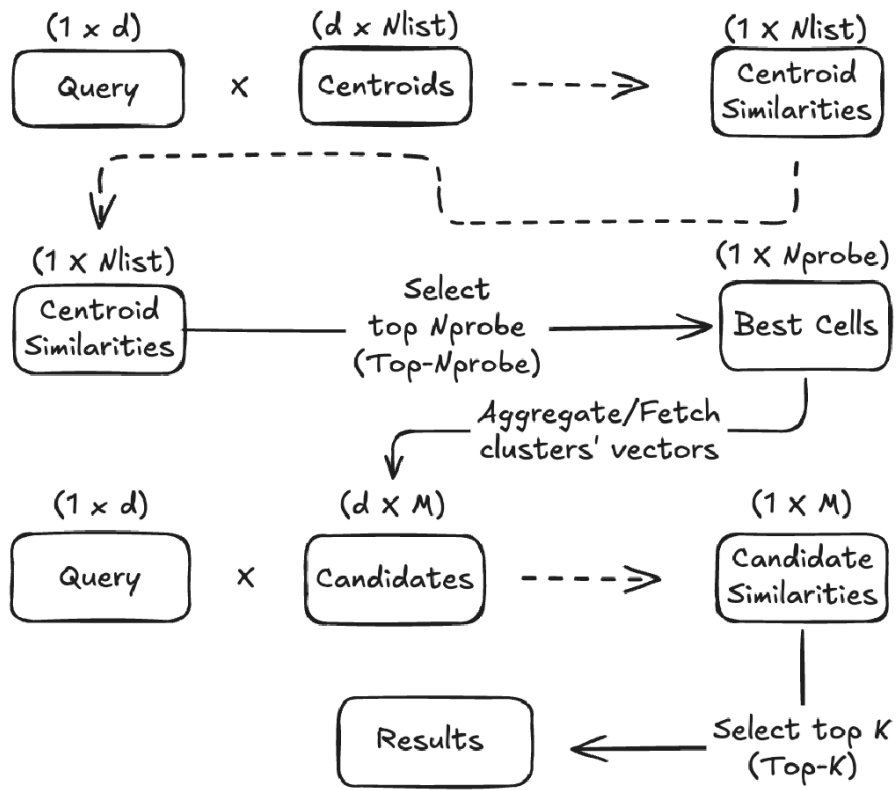


Figure 3.1. Logical IVF-Flat query-processing pipeline. The query is first compared with all N_{list} centroids. The N_{probe} best cells are selected, their vectors define the candidate set, and the final top- k results are selected from the candidate similarities.

the implementation uses

$$B = \left\lceil \frac{Q}{32} \right\rceil \quad (3.1)$$

internal query batches. The final batch is padded when Q is not divisible by 32. The number of valid rows is retained separately, so padded rows do not participate in coarse-result processing and do not contribute inverted lists to the fine-search schedule. They are also discarded when the final results are converted back to the host representation.

Although the implementation is described throughout the chapter in terms of one 32-query batch for clarity, the final system can process multiple such batches within one coarse-search and one fine-search invocation. The transition from the initial single-batch implementation to this multi-batch design is discussed in Section 3.6.

3.1.2 Host–Device Work Partitioning

The index data required by both search stages remain resident on the Tenstorrent device across query workloads, including the centroid matrix, database vectors grouped by inverted list, and the corresponding original vector identifiers. In contrast, query-dependent data, such as the query buffer, coarse-search outputs, selected-list metadata, worker schedules, partial results, and final outputs, are updated for each search workload.

A synchronization boundary separates the coarse and fine programs. After the device executes coarse search and returns a fixed-size set of centroid candidates for each valid query, the host performs the final N_{probe} selection, obtains the offset and padded length of each corresponding complete inverted list, groups the selected work at batch granularity, and constructs the per-worker fine-search descriptions. Only once this query-dependent schedule has been prepared is the fine-search program launched.

Figure 3.2 shows the division of responsibilities between the host and the device, together with the synchronization boundary between coarse and fine search.

Each device program is decomposed into *reader*, *compute*, and *writer* kernels connected through circular buffers, allowing data movement and computation to overlap when dependencies and buffer capacity permit it. Throughout this chapter, reader, compute, and writer denote kernels executing on a Tensix core, whereas worker and aggregation core denote logical roles assigned to complete Tensix cores. The terms *coarse phase* and *fine phase* refer to complete TT-Metalium programs rather than to individual kernels.

3.2 Device-Oriented IVF Representation

3.2.1 Index Construction

Index training is treated as an offline preprocessing step and is outside the timed search path. A representative subset of the database vectors is clustered with Faiss k -means to obtain N_{list} centroids [3], following the same training procedure used by the CPU and GPU baselines based on Faiss `IndexIVFFlat`. The trained centroid matrix is exported and reused by the Tenstorrent implementation. Reusing the same

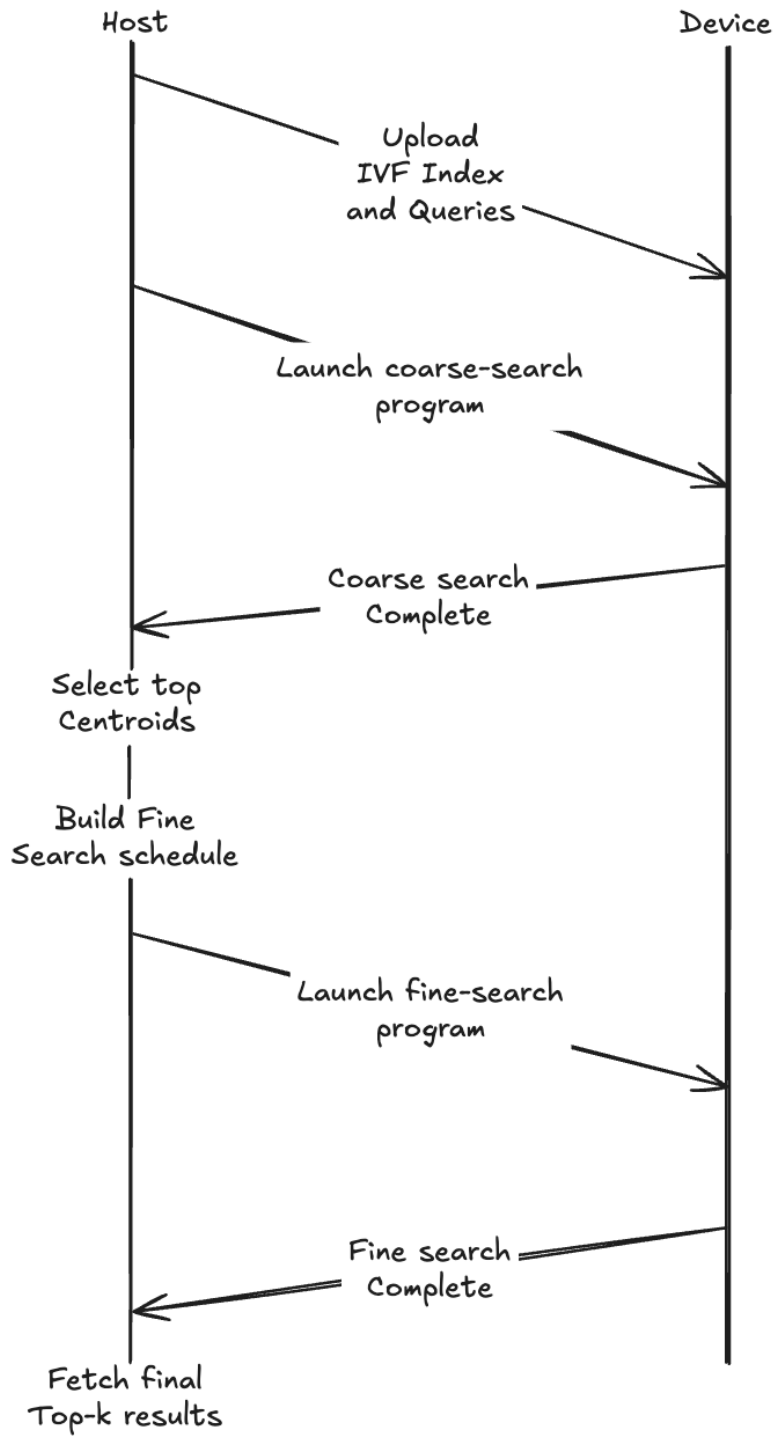


Figure 3.2. Host-device interaction during one IVF search workload. The host loads the IVF index and query batch, launches the coarse-search program, receives the coarse-search results, selects the N_{probe} lists and builds the fine-search schedule, then launches the fine-search program and fetches the final top- k results.

centroids avoids making index-training quality an additional experimental variable and allows the evaluation to focus on differences in search execution.

Training is not performed on the Tenstorrent device because the objective of this work is the online IVF search path rather than accelerator-based k -means. An on-device training implementation would require additional iterative kernels for vector assignment, per-cluster accumulation, centroid normalisation, and convergence control, together with temporary device-memory capacity for the intermediate state. Those operations would neither be part of the measured query path nor improve the fairness of the search comparison. During construction of the Tenstorrent-oriented index, each database vector is assigned to its closest exported centroid and placed in the corresponding packed inverted list.

The Tenstorrent index stores full, uncompressed database vectors rather than compressed representations. Database vectors and centroids are normalised before assignment and search, allowing inner product to be used as the similarity measure.

Because the resulting inverted lists are not guaranteed to have equal sizes, assigning the same number of lists to every core does not necessarily assign the same amount of work. The fine-search scheduler therefore reasons about the number of 32-vector pages contained in each selected list rather than only the number of lists.

3.2.2 Tiled Data Layout and Memory Placement

As introduced in Chapter 2, Tensix computation is organised around 32×32 tiles [11]. Let D denote the original vector dimension. To match this tile-oriented execution model, the dimension is padded to

$$D_p = 32 \left\lceil \frac{D}{32} \right\rceil. \quad (3.2)$$

All values introduced by dimensional padding are set to zero on the host before the data are transferred to the device.

Vectors assigned to the same inverted list are stored consecutively in one logical packed database buffer. For a list containing L_j valid vectors, its stored length is rounded to a multiple of 32:

$$L_{j,p} = 32 \left\lceil \frac{L_j}{32} \right\rceil. \quad (3.3)$$

Each group of 32 vectors forms one *candidate page*. A page contains $D_p/32$ 16-bit floating-point (FP16) data tiles, each of shape 32×32 , which together represent the 32 candidate vectors over the complete padded dimension. Centroids and queries use the same FP16 representation, while vector identifiers are stored as 32-bit unsigned integers (UInt32). The Tenstorrent compute configuration enables FP32 destination accumulation. This mode is required by the employed top- k path when score tiles are ordered together with UInt32 identifier tiles.

A candidate page must not be confused with a query batch. A query batch contains up to 32 queries and forms the row operand of fine search, whereas a candidate page contains 32 database vectors and forms the column operand. Their matrix product evaluates every query in the batch against every vector in the page. Figure 3.3 illustrates this distinction with a 4×4 example.

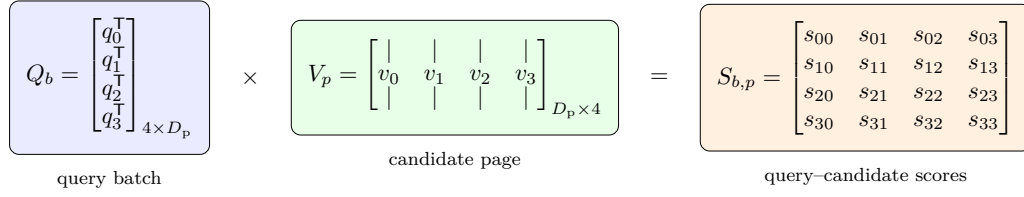


Figure 3.3. Pedagogical 4×4 example of one fine-search matrix multiplication. Rows represent queries and columns represent candidate vectors. The implementation uses 32×32 tiles rather than the reduced size shown here.

For every candidate page, one full 32×32 `UInt32` identifier tile is stored. In its DRAM representation, each column corresponds to one candidate vector and the same 32 identifiers are replicated across all query rows. For candidate identifiers $i_0, \dots, i_{31}$, the stored tile is

$$I^{\text{stored}} = \begin{bmatrix} i_0 & i_1 & \cdots & i_{31} \\ i_0 & i_1 & \cdots & i_{31} \\ \vdots & \vdots & \ddots & \vdots \\ i_0 & i_1 & \cdots & i_{31} \end{bmatrix}. \quad (3.4)$$

Before the compute-kernel `topk_local_sort` operation, both the similarity tile and its companion identifier tile are transposed. The identifier tile presented to the selection operation is therefore

$$I^{\text{topk}} = (I^{\text{stored}})^T = \begin{bmatrix} i_0 & i_0 & \cdots & i_0 \\ i_1 & i_1 & \cdots & i_1 \\ \vdots & \vdots & \ddots & \vdots \\ i_{31} & i_{31} & \cdots & i_{31} \end{bmatrix}. \quad (3.5)$$

After transposition, each column corresponds to one query and the rows represent candidate entries. The `topk_local_sort` operation sorts candidate entries independently for each query column. Applying the same transpose and ordering operations to the score and identifier tiles preserves the association between every similarity value and its original database-vector identifier. This transpose-and-sort mechanism is used by both coarse and fine search whenever a running top-32 score and identifier state is updated.

The vectors of all inverted lists are stored sequentially in a single packed device buffer. Each inverted list is identified by its offset within this buffer and by its length, rather than being stored in a separate device allocation. Vector identifiers occupy a separate packed `UInt32` buffer using the same list and page ordering. An inverted-list offset therefore identifies a position in this global logical representation. At the physical memory level, TT-Metalium interleaves buffer pages across DRAM banks, so “contiguous” refers to the logical buffer layout rather than to consecutive bytes within one DRAM bank.

Persistent search data are allocated in device DRAM, including the centroid matrix, packed database-vector buffer, and packed vector-identifier buffer. Query buffers and worker descriptions are likewise allocated in DRAM for each search

workload, while per-core L1 memory is used as transient kernel-local storage through circular buffers. During execution, data required by the active kernels are streamed from DRAM into L1 rather than persistently cached across search invocations. The exact reuse and streaming strategy of the coarse- and fine-search programs is described in Sections 3.3 and 3.5.1, respectively.

Padding is performed entirely on the host before device transfer:

- database dimensional padding and inverted-list length padding are generated during `IVF_TT::create_index`.
- centroid dimensional padding is generated while constructing the centroid buffer in `load_to_device`.
- query dimensional padding and final-batch row padding are generated in `search_all`.

Padded vector identifiers are assigned the sentinel value `0xffffffff` on the host.

Padded candidate positions contain zero-valued vectors and carry the sentinel identifiers defined above. Before local top- k selection, the fine-search compute kernel linearly scans the score and identifier entries of the final partially populated page of each inverted list. Whenever a sentinel identifier is encountered, the corresponding similarity value is replaced with -10000 , preventing padded candidates from displacing valid results during selection. The database and query vectors are normalised, so a valid inner product is bounded below by -1 in exact arithmetic; -10000 is therefore safely outside the valid similarity range. Complete candidate pages contain 32 valid vectors and bypass this masking step, avoiding an unnecessary scan of all $32 \times 32 = 1024$ entries when no padding is present.

3.3 Coarse Search

The purpose of coarse search is to identify the inverted lists most relevant to each query. For a valid query batch $Q_b \in \mathbb{R}^{32 \times D_p}$ and centroid matrix $C \in \mathbb{R}^{D_p \times N_{\text{list}}}$, the logical operation is

$$S_b^{\text{coarse}} = Q_b C, \quad (3.6)$$

where each row of S_b^{coarse} contains the similarities between one query and all centroids.

Tiling proceeds over blocks of 32 centroids. After loading the query batch from DRAM into an L1 circular buffer, the reader kernel streams centroid blocks from device DRAM while retaining the query for reuse across the complete centroid matrix. Let $C_c \in \mathbb{R}^{D_p \times 32}$ denote centroid block c .

The corresponding coarse-similarity tile is

$$S_{b,c}^{\text{coarse}} = Q_b C_c, \quad S_{b,c}^{\text{coarse}} \in \mathbb{R}^{32 \times 32}. \quad (3.7)$$

Rather than materialising all N_{list} similarities in DRAM, the compute kernel incrementally maintains the 32 highest centroid scores and their identifiers for each query row. This running top-32 state uses the same transpose-and-topk_local_sort mechanism described in Section 3.2.2, with identical ordering decisions applied to scores and centroid identifiers.

Once all centroid blocks assigned to a query batch have been processed, the coarse-search writer kernel running on the same participating Tensix core writes one score tile and one identifier tile for that batch to device DRAM. After copying these results to the host, the host orders and deduplicates the valid centroid identifiers and retains the first N_{probe} lists for each query. Consequently, the current implementation limits N_{probe} to at most 32.

Multiple query batches are distributed by the coarse-search program across the available participating cores, with each active core processing one or more complete batches and avoiding inter-core communication during the centroid scan. Because every valid query is evaluated against the same centroid matrix, coarse search remains regular and requires a predictable amount of work.

3.4 Fine-Search Scheduling

Fine search introduces the principal source of irregularity in the system. Every valid query selects the same number N_{probe} of inverted lists, but the identifiers and lengths of those lists depend on the query. Two queries with the same N_{probe} may therefore require very different amounts of candidate processing. The imbalance remains after lists are combined at batch level: one worker assigned several short lists can finish before another worker assigned one very long list.

The scheduler estimates the cost of a selected list by its number of padded 32-vector candidate pages. This page count is used only as a scheduling weight: an inverted list remains an indivisible task and is never partitioned into page ranges shared by multiple workers.

3.4.1 Batch-Level List Union

After coarse search, each query row has up to N_{probe} selected inverted-list identifiers. Since fine search operates on a complete batch of up to 32 queries, the host collects these identifiers into a single set of lists for the batch.

Let $\mathcal{P}_{b,r}$ denote the N_{probe} lists selected for query row r of batch b . The set of lists processed for the batch is

$$\mathcal{U}_b = \bigcup_{r \in \mathcal{V}_b} \mathcal{P}_{b,r}, \quad (3.8)$$

where $\mathcal{V}_b$ contains the valid query rows of the batch.

Operationally, the host iterates over the coarse-search results of every valid query row and inserts the first N_{probe} list identifiers into a set. Duplicate identifiers are therefore automatically removed. The resulting set $\mathcal{U}_b$ contains every inverted list selected by at least one query in the batch.

The batch-level union is motivated by the execution granularity of the Tensix matrix engine. Fine search operates on 32×32 tiles: one candidate page contains 32 vectors, while one query tile contains up to 32 queries. For a candidate page V_p , the device evaluates

$$Q_b V_p, \quad Q_b \in \mathbb{R}^{32 \times D_p}, \quad V_p \in \mathbb{R}^{D_p \times 32}, \quad (3.9)$$

producing a 32×32 similarity tile.

Because Tensix executes the matrix multiplication at tile granularity, processing a candidate page for only one query row does not reduce the cost of the matrix operation relative to processing it for the complete query tile. A computation conceptually involving a $1 \times D_p$ query would still occupy the same tiled execution unit after padding to the required representation. Therefore, once a candidate page is selected for processing, evaluating it against the remaining queries of the batch does not require an additional matrix multiplication.

As a consequence, a query can also obtain similarity scores for vectors belonging to a list that was selected by another query in the same batch. The effective candidate set for one query can therefore be larger than the set defined by its own N_{probe} lists. This is an implementation consequence of processing the batch-level union and does not modify the stored IVF index.

The union can nevertheless increase the total amount of fine-search work because lists selected by different queries may not overlap, increasing the number of distinct candidate pages that must be processed. The benefit of the design therefore comes from fully exploiting each tile-level matrix multiplication, rather than from making the additional lists computationally free.

3.4.2 Greedy Work Distribution

For each list $j \in \mathcal{U}_b$, the host creates one indivisible task that covers the entire inverted list. Its scheduling weight is the number of padded candidate pages in that list,

$$p_j = \left\lceil \frac{L_j}{32} \right\rceil, \quad (3.10)$$

where L_j is the number of valid vectors in list j . Thus, for example, lists containing 320 and 288 vectors have weights 10 and 9, respectively. These weights count the fixed-shape page multiplications expected for each list; they are not fractions used to divide a list among workers.

The implementation configures an 8×8 logical grid of 64 Tensix cores. Logical core (0,0) is reserved for final aggregation, and all remaining 63 cores form the fine-search worker set $\mathcal{W}$. For every query batch, let $\mathcal{A}_{b,w}$ denote the lists assigned to worker w . The scheduler resets both the assignments and their estimated loads,

$$\mathcal{A}_{b,w} = \emptyset, \quad \lambda_{b,w} = 0, \quad w \in \mathcal{W}, \quad (3.11)$$

sorts the tasks in non-increasing order of p_j , and assigns each complete list to a currently least-loaded worker:

$$w^* = \arg \min_{w \in \mathcal{W}} \lambda_{b,w}, \quad (3.12)$$

$$\mathcal{A}_{b,w^*} \leftarrow \mathcal{A}_{b,w^*} \cup \{j\}, \quad (3.13)$$

$$\lambda_{b,w^*} \leftarrow \lambda_{b,w^*} + p_j. \quad (3.14)$$

The secondary ordering key for equal page counts is ascending inverted-list identifier. If several workers have the same minimum load, the worker with the lowest index is selected. These explicit tie rules make the host schedule reproducible. The resulting

longest-task-first greedy policy attempts to balance the padded page count while retaining sequential access within every packed list.

Figure 3.4 shows the process for a reduced four-query example. The displayed values 10, 9, 6, and 4 are page-count weights. In particular, a list of weight 10 remains one task; it is not split into ten independently assignable ranges.

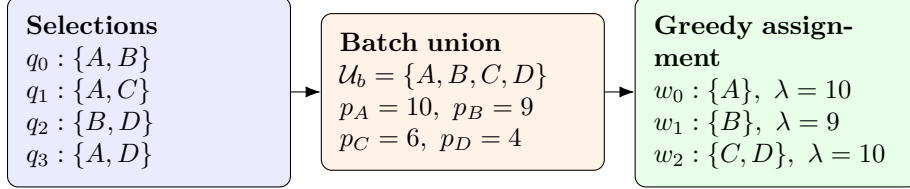


Figure 3.4. Pedagogical four-query example of batch-level list union and greedy whole-list scheduling. The real implementation uses batches of up to 32 queries and all configured fine-search workers. Each selected list is an indivisible task weighted by its padded number of candidate pages.

The page count is an estimate rather than a cycle-accurate cost model. It does not capture per-list setup, masking of the final partial page, variation in memory service time, circular-buffer waiting, or synchronization. Moreover, because a large inverted list cannot be divided, the longest list can still determine the batch completion time. After scheduling, the host serialises one description per fine-search worker so that its reader kernel can locate the assigned complete lists in device memory.

3.4.3 Worker Description

The host serialises the fine-search assignment into one logical flattened worker description in DRAM for each fine-search worker. A description can span multiple DRAM pages, so it is not restricted to one physical page. It contains the information required by the reader kernel to locate the query batches and the complete inverted lists assigned to that worker.

Let B denote the number of query batches, as introduced in Section 3.1.1. The worker description has the conceptual form

$$W_w = \langle B, R_0, R_1, \dots, R_{B-1} \rangle, \quad (3.15)$$

where R_b describes the work assigned to worker w for query batch b .

Each batch record contains the global offset of the query batch in the query buffer, the number of tiles used to represent it, and the number of whole-list tasks that the worker must process:

$$R_b = \left\langle o_b^Q, n_b^Q, N_{\text{list},b}^{(w)}, E_{b,0}^{(w)}, E_{b,1}^{(w)}, \dots, E_{b,N_{\text{list},b}^{(w)}-1}^{(w)} \right\rangle, \quad (3.16)$$

where o_b^Q is the global offset of query batch b in the query buffer, n_b^Q is the number of tiles composing its padded representation, and $N_{\text{list},b}^{(w)}$ is the number of whole-list tasks assigned to worker w for that batch.

Each whole-list entry r is represented as

$$E_{b,r}^{(w)} = \langle o_{b,r}^V, n_{b,r}^V \rangle, \quad (3.17)$$

where $o_{b,r}^V$ is the global candidate-page offset of the assigned inverted list in the packed database buffer, and $n_{b,r}^V$ is the number of candidate pages in that complete list. Although the record is encoded as an offset and page-count pair, it always describes an entire selected list; the scheduler does not create subranges. Since each candidate page contains 32 vectors, these two values are sufficient for the reader kernel to determine where the list begins and how many candidate-vector and identifier tiles must be fetched from their respective packed buffers.

At the beginning of fine search, each worker copies its description from DRAM into local L1. For each batch with assigned work, the corresponding query tiles are loaded once into the local query circular buffer and remain resident while all whole-list tasks associated with that batch are processed. The reader then advances to the next batch record and repeats the same procedure.

3.5 Fine Search and Result Reduction

3.5.1 Worker Pipeline

The fine-search program evaluates the candidate pages selected by the host schedule. As defined above, logical core (0,0) performs final aggregation, while the other 63 cores in the configured 8×8 logical grid operate as fine-search workers.

Each worker executes reader, compute, and writer kernels concurrently. At the beginning of the invocation, the reader loads the worker description from DRAM into local L1; for each active query batch, the query tiles are then loaded once into the worker’s query circular buffer and retained while all candidate pages assigned to that worker for the batch are processed.

Candidate vectors and their identifier tiles are streamed from the packed DRAM buffers. The reader issues both reads before a shared synchronization barrier, allowing their transfers over the on-device Network-on-Chip (NoC) to be outstanding concurrently. Double-buffering of the candidate circular buffers allows the reader to prepare a later page while the compute kernel operates on an earlier one, subject to circular-buffer capacity and producer–consumer synchronization.

Let j identify a whole inverted-list task assigned to the worker and let p identify a candidate page within that list. For one such page $V_{j,p} \in \mathbb{R}^{D_p \times 32}$, the compute kernel evaluates

$$S_{b,j,p}^{\text{fine}} = Q_b V_{j,p}, \quad (3.18)$$

producing a 32×32 tile of query–candidate similarities.

Across all candidate pages assigned to it, the worker incrementally maintains a row-wise top-32 state, with scores and original vector identifiers propagated together through the same ordering operations. No list-local to global identifier reconstruction is therefore required after the search.

Figure 3.5 shows the local top-32 update performed by one fine-search worker. Two 32×32 tiles hold the best scores and identifiers found so far, while the other two contain the scores and identifiers of the current candidate page. Before selection, the current and saved score and identifier tiles are transposed so that each column corresponds to one query and the rows contain candidate entries. The compute-kernel `topk_local_sort` operation then applies the same ordering decisions to the

score and identifier tiles, preserving the association between every similarity value and its original vector identifier.

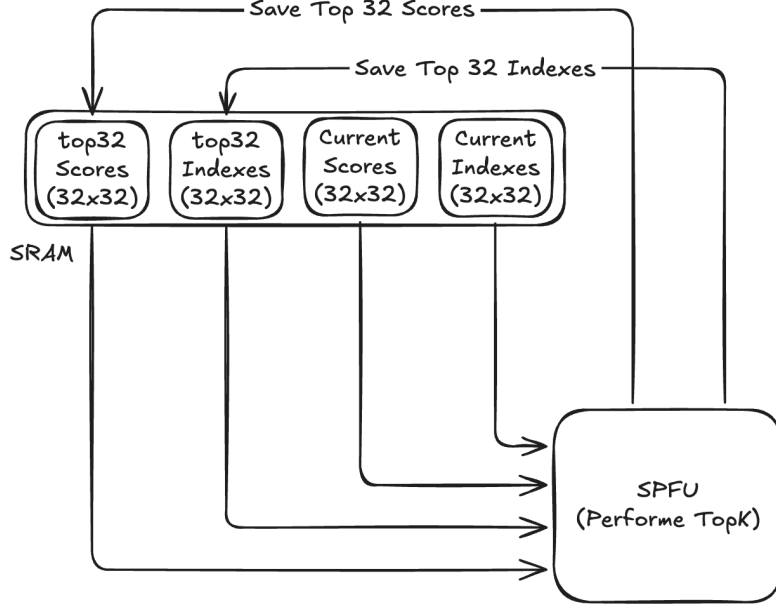


Figure 3.5. Worker-local top-32 update. Before selection, the score and identifier tiles are transposed so that each column corresponds to one query and the rows contain candidate entries from the saved and current states. The `topk_local_sort` operation applies identical ordering decisions to scores and identifiers.

3.5.2 Hierarchical Top- k

A single worker observes only part of the candidate set, so the final result is computed hierarchically. For query row r in batch b , worker w first retains

$$R_{b,w,r} = \text{Top}_{32}(\mathcal{C}_{b,w,r}), \quad (3.19)$$

where $\mathcal{C}_{b,w,r}$ is the set of candidates evaluated by that worker.

Worker-local results are staged in DRAM, with each configured worker writing one fixed-size score tile and one identifier tile per query batch to a region indexed by batch and worker. All configured workers produce and signal one partial result for every batch, even when a worker has no active list task for that batch. In that case, every score remains at the initial value -10000 and every identifier remains at `0xffffffff`.

Synchronization uses a cumulative completion semaphore in the L1 memory of logical core $(0,0)$. After writing its partial result for a batch, each worker increments this semaphore, and the aggregation reader waits until the expected number of increments has been observed. Once the partial result of a worker has been consumed, the aggregation reader increments a worker-specific ready semaphore in that worker's L1 memory, allowing the worker to reuse its partial-result slot and advance to the next batch.

Figure 3.6 illustrates this two-way handshake. Fine-search workers write their local top-32 results to distinct locations in the shared DRAM partial-result buffer and signal completion to the aggregation core; after consuming those results, core (0,0) signals the corresponding workers that their slots can be reused.

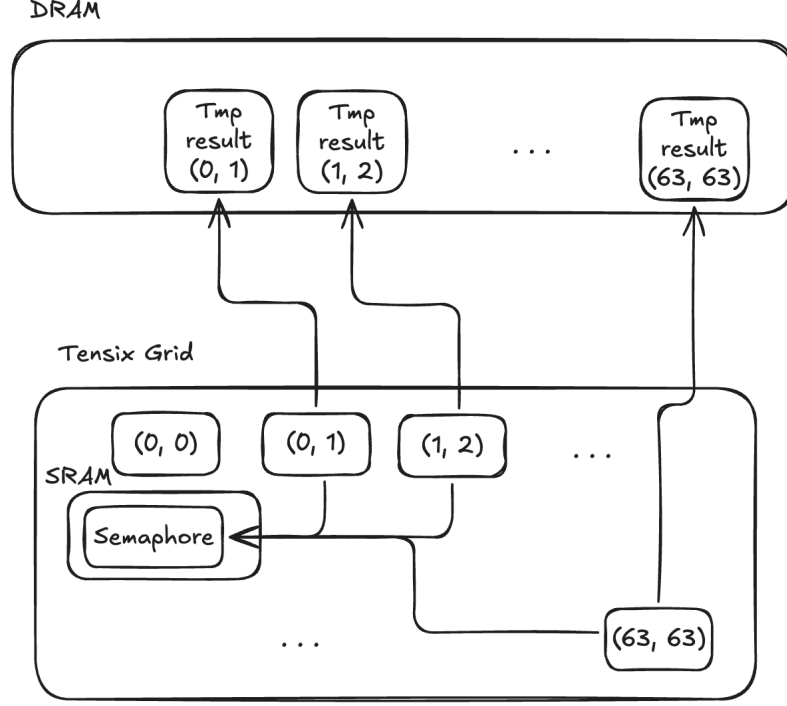


Figure 3.6. DRAM staging and synchronization of fine-search partial results. Workers store their local top-32 results in DRAM and synchronize with the aggregation core through completion and ready semaphores.

The aggregation compute kernel performs the second reduction,

$$R_{b,r} = \text{Top}_{32} \left(\bigcup_w R_{b,w,r} \right), \quad (3.20)$$

and writes one final score tile and one final identifier tile per batch to DRAM. Figure 3.7 reduces the tile width from 32 to 4 to illustrate the two-level selection without the visual complexity of the actual result tiles.

The host converts the tiled outputs to row-major form, removes sentinel entries, discards padded query rows, and retains the first k results for every valid query. The implementation therefore supports $k \leq 32$; the evaluation in this thesis uses $k = 10$.

For B query batches and W fine-search workers, logical core (0,0) consumes BW worker-result pairs and performs BW running top-32 updates. Synchronization activity and partial-result traffic also grow linearly with the number of worker results. With $W = 63$, each batch contributes 63 score-and-identifier result pairs to the aggregation stage and requires 63 running top-32 updates. Because workers without active list tasks still write sentinel results, this cost depends on the configured worker count rather than the number of non-empty assignments. The centralised stage consequently scales as $\Theta(BW)$ and can constrain designs with larger worker counts.

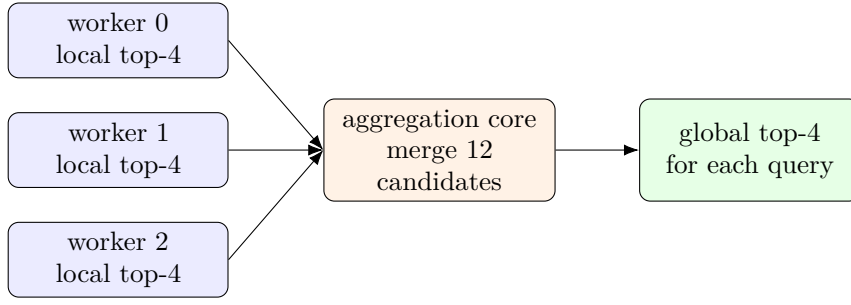


Figure 3.7. Pedagogical two-level result reduction for three workers, retaining four entries. Each worker first retains its local best four candidates; logical core (0,0) merges the partial results and retains the global best four for each query. The implementation uses top-32 states and all configured workers.

3.6 Multi-Batch Execution

The first implementation processed one batch of at most 32 queries per search invocation. This design matched the 32×32 tile representation and simplified the initial implementation, but a larger query workload required the host to repeat query preparation, coarse execution, synchronization, list selection, fine-search scheduling, fine execution, final result transfer, and host-side orchestration for every group of 32 queries.

The final implementation, represented by `ann_ivf_dram_args`, retains the same 32-query internal representation while allowing one search call to process multiple batches. After a coarse-search invocation processes all valid batches of the workload, the host constructs the fine-search descriptions for the complete set of selected lists. These logical descriptions may occupy multiple DRAM pages, so the format does not impose a fixed batch-count limit. The fine-search program then processes the paged descriptions within one larger invocation; practical capacity remains bounded by the device-memory allocation available to the workload.

This change separates tile size from execution granularity: a batch of 32 queries remains the natural unit of matrix computation, but it is no longer the unit at which the complete device pipeline must be launched. Repeated program configuration, launch, synchronization, and host orchestration are consequently amortised across a larger number of queries.

3.7 Implementation Constraints

Table 3.1 consolidates the implementation constraints derived in the preceding sections without repeating their full derivations.

Reader, compute, and writer kernels additionally share the local L1 capacity through their circular buffers, query tiles, intermediate similarities, identifier tiles, running top-32 states, and worker metadata. Buffer depth must therefore balance query reuse and double buffering against local-memory capacity. Progress also remains subject to circular-buffer backpressure, NoC completion, and the synchronization dependencies described above.

Table 3.1. Principal constraints of the evaluated Tenstorrent implementation.

Constraint	Cause	Consequence
Tile alignment	The compute path operates on 32×32 tiles.	Vector dimensions and list lengths are padded to multiples of 32; an internal query batch contains up to 32 valid queries.
Centroid blocking	Coarse search traverses complete 32-centroid blocks.	The current representation requires N_{list} to be divisible by 32.
Selection capacity	Coarse, worker-local, and global selection use top-32 states.	$N_{\text{probe}} \leq 32$ and $k \leq 32$. The reported experiments use $k = 10$.
Whole-list scheduling	A worker record addresses one complete inverted list.	Lists cannot be split across workers, so a large list can determine the batch tail even after greedy balancing.
Paged worker descriptions	A logical description may span multiple DRAM pages.	The format imposes no fixed batch-count limit; practical capacity depends on the device-memory allocation available to the invocation.
Centralised reduction	Every worker emits one partial result for every batch.	Aggregation work and intermediate traffic scale as $\Theta(BW)$, including sentinel results from workers with empty assignments.
Deployment scope	The implementation targets one Wormhole n150 device.	Multi-device approximate nearest-neighbour execution is outside the scope of this work.

3.8 Chapter Summary

This chapter described the implementation of IVF-Flat search on a single-chip Tenstorrent Wormhole n150 device, with the index transformed into a tile-oriented representation in which vectors are grouped by inverted list and stored in 32-vector pages together with their original identifiers.

The search pipeline is divided into a regular coarse-search phase and an irregular fine-search phase. While coarse search reuses query tiles to scan the centroid matrix and return a fixed-size set of centroid candidates, the host subsequently selects the N_{probe} lists, forms batch-level unions, and assigns complete selected lists to workers using their padded page counts as weights in a greedy longest-task-first scheduler.

Each fine-search worker reuses one query batch while streaming the candidate pages of its assigned lists and maintaining a worker-local row-wise top-32 state. The resulting partial results are staged in DRAM and merged on logical core (0, 0) before the final outputs are returned to the host.

The final design preserves the natural 32-query tile representation while extending execution to multiple batches per invocation. This reduces repeated host-device orchestration while preserving the same coarse- and fine-search stages and using batch-level list unions to match the tile-oriented execution model. The following chapter defines the methodology used to evaluate the resulting implementation.

Acknowledgements

Bibliography

- [1] GUO, R., SUN, P., LINDGREN, E., GENG, Q., SIMCHA, D., CHERN, F., AND KUMAR, S. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning*, vol. 119 of *Proceedings of Machine Learning Research*, pp. 3887–3896. PMLR (2020).
- [2] IWASAKI, M. Pruned bi-directed k-nearest neighbor graph for proximity search. In *Similarity Search and Applications*, vol. 9939 of *Lecture Notes in Computer Science*, pp. 20–33. Springer (2016). doi:10.1007/978-3-319-46759-7_2.
- [3] JOHNSON, J., DOUZE, M., AND JÉGOU, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, **7** (2021), 535. doi:10.1109/TBDATA.2019.2921572.
- [4] MALKOV, Y. A. AND YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **42** (2020), 824. doi:10.1109/TPAMI.2018.2889473.
- [5] MILVUS. Hnsw. <https://milvus.io/docs/hnsw.md>. Accessed: 2026-07-30.
- [6] MILVUS. Ivf-pq. <https://milvus.io/docs/ivf-pq.md>. Accessed: 2026-07-30.
- [7] OOTOMO, H., NARUSE, A., NOLET, C., WANG, R., FEHER, T., AND WANG, Y. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for gpus. In *2024 IEEE 40th International Conference on Data Engineering*, pp. 4236–4247. IEEE (2024). doi:10.1109/ICDE60146.2024.00323.
- [8] ORACLE. Understand inverted file flat vector indexes. <https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/understand-inverted-file-flat-vector-indexes.html>. Accessed: 2026-07-30.
- [9] PUGH, W. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, **33** (1990), 668. doi:10.1145/78973.78977.
- [10] SHI, J., GAO, J., XIA, J., FEHÉR, T. B., AND LONG, C. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *arXiv preprint arXiv:2602.23999*, (2026).

- [11] TENSTORRENT. Tt-metalium documentation: Architecture and programming model. <https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/>. Accessed: 2026-07-30.
- [12] TENSTORRENT. Wormhole pcie cards: Hardware specifications. <https://docs.tenstorrent.com/aibs/wormhole/index.html>. Accessed: 2026-08-26.